

biobouncer

A gate for biological inputs

Samuel Bharti

The regex you already have

Every app has one. None of them agree.

```
is_gene <- function(x) grepl("^[A-Z0-9-]+$", x)

is_gene(c("MLL", "KMT2A", "mondo:5148"))
#> [1] TRUE TRUE FALSE
```

MLL passes

Retired in 2013. The gene is **KMT2A**.

shape is not existence

mondo:5148 fails

One uppercase and three zeros from a real id.

the user meant MONDO:0005148

The regex knows the shape and nothing else.



One gate, three languages

50

IDENTIFIER SOURCES

4

VALIDATION MODES

3

LANGUAGES, ONE CORPUS

pattern

cache

remote

existence

CRAN

PyPI

npm soon

pattern

is it well formed?
regex or grammar

offline

cache

does it exist?
against a dated snapshot

offline

remote

does it exist right now?
the live source API

online

existence

snapshot first,
remote only without one

either

cheapest, deterministic, belongs in tests

most expensive, needs a network

check_id(): one row per input

```
check_id(c("MONDO:0005148", "mondo:5148", "GO:0006915"), "mondo")
#>   input          valid normalized suggestion source_db how
#>   MONDO:0005148 TRUE   MONDO:0005148 NA        mondo    pattern
#>   mondo:5148    FALSE NA          MONDO:0005148 mondo    pattern
#>   GO:0006915    FALSE NA          NA         mondo    pattern
```

valid

Only ever TRUE, FALSE, or NA for a missing input.

suggestion

What the user probably meant. Never invented.

version, how

Which snapshot and which mode gave this verdict.

Same length, same order as the input. It sits next to your data frame.

Repair a messy table

```
df <- read.csv("data/associations.csv")
report_id(df$gene, "hgnc", how = "cache")
#> 7 values: 4 valid, 2 repairable, 1 invalid, 1 missing

df$gene <- repair_id(df$gene, "hgnc", how = "cache")
#> MLL -> KMT2A, CARS -> CARS1, notagene unchanged
```

`report_id()`

One line summary per column.

`check_id()`

→ The rows behind the summary.

`repair_id()`

→ Substitute the fixable, leave the rest.

Repair, do not drop. A user who typed MLL meant KMT2A.

Snapshots have dates

```
biobouncer_snapshots()
#>   source version      n_ids location
#>   hgnc    2026-07-07  45019 bundled
#>   hgnc    sample           30 bundled
#>   mondo   sample           25 bundled

biobouncer_pull("mondo")      # once, online
check_id(x, "mondo", how = "cache", version = "2026-09-01")
```

Pull once

A snapshot is a file with a date in its name.

Check forever

Name the version. Re-run in a year, same verdict.

The version string goes in your methods section.

It drops into what you already use

checkmate

assertr

validate

pointblank

shinyvalidate

pandera

pydantic

```
assert_valid_id(ids, "hgnc", how = "cache")      # throws at the top of a function
is_hgnc <- id_predicate("hgnc", how = "cache")    # element-wise, for assertr
iv$add_rule("gene", sv_biobouncer("hgnc", how = "cache")) # a Shiny input
```

Every adapter calls the same classifier. None of them re-implements a rule.

Same answer in Python

R

```
check_id(c("mondo:5148"), "mondo")  
#> valid FALSE  
#> suggestion MONDO:0005148  
  
report_id(genes, "hgnc", how = "cache")  
#> 2 repairable
```

PYTHON

```
bb.check_id(["mondo:5148"], "mondo")  
#> valid False  
#> suggestion MONDO:0005148  
  
bb.report(genes, "hgnc", how="cache")  
#> 2 repairable
```

One conformance corpus. Two implementations. Zero drift.

Start here

Public

```
install.packages("biobouncer")
```

```
pip install biobouncer
```

Docs: samuelbharti.com/biobouncer

Try it in a minute

```
library(biobouncer)
check_id("mondo:5148", "mondo")
repair_id(c("MLL", "CARS"), "hgnc", how = "cache")
```

Tutorial: [tutorials/biobouncer.qmd](#) in this repo.